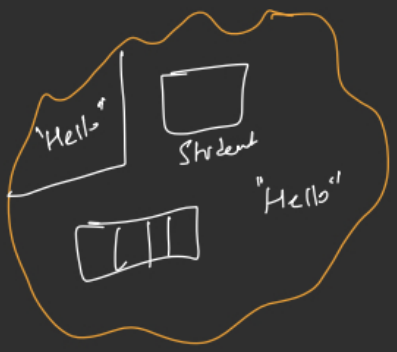
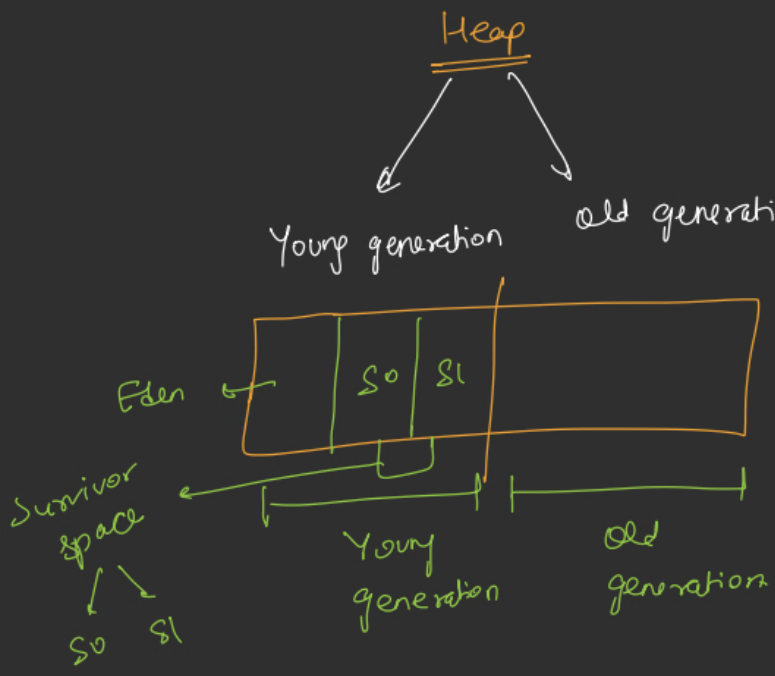
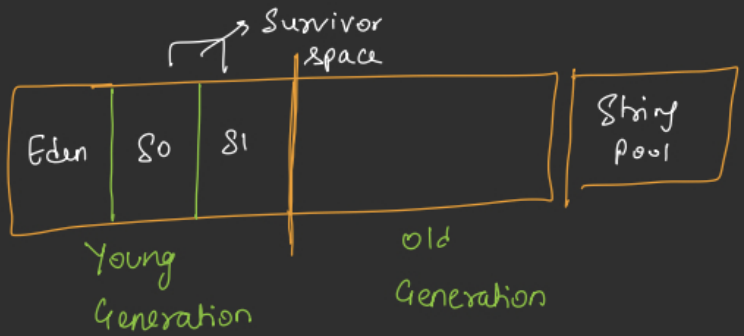
Heap Memory



new

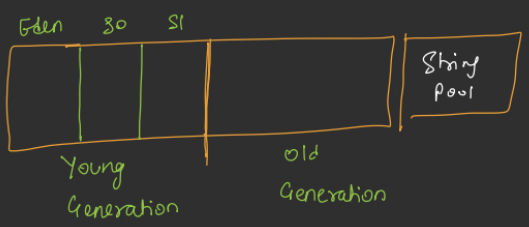




Student S1 = new Student();

Eden

↳ Allocation is very fast



X [Ob1] [Ob2] [Ob3] [Ob4] → Eden

→ Young Gen → Minor GC

① After 1st GC.
[Ob2] [Ob4] → S0

② After 2nd GC
[Ob2] [Ob4] → S1

③ After 3rd GC.
[Ob2] [Ob4] → S0
X

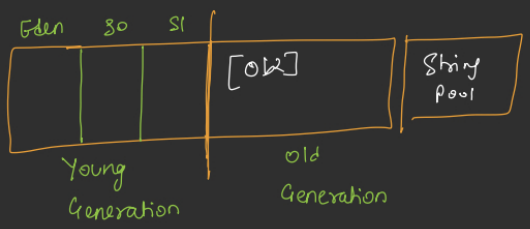
⋮

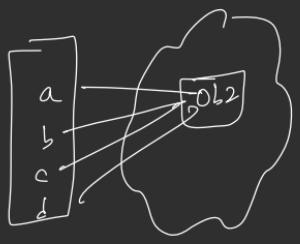
⑮ After 15th GC
[Ob2] → Old Generation

[Ob2] age X ~~2~~ ~~3~~ 15
[Ob4] age X ~~2~~ ~~3~~ ~~13~~

Old Generation

↳ Major GC runs in this memory.
↳ expensive.





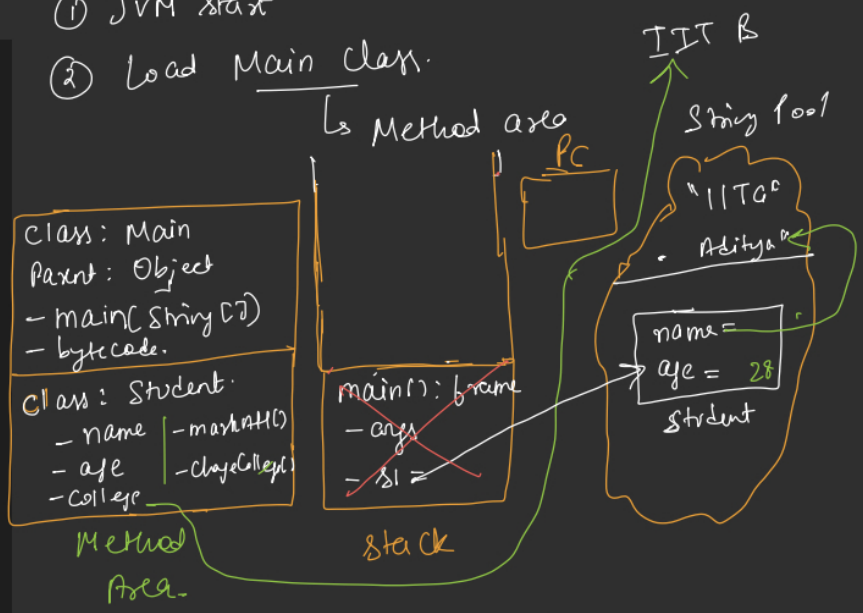
Counter = 15

① Survivor Space Overflow.

② Dynamic age determination.

③ Large Object
int[] arr = new int[10_000_000];

```java
class Student {
    String name;
    int age;
    static String college = "IIT G";

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Instance method (normal method)
    public void markAttendance() {
        System.out.println(name + " is present");
    }

    // Static method
    public static void changeCollege(String newCollege) {
        college = newCollege;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Aditya", 28);

        s1.markAttendance();

        Student.changeCollege("IIT Bombay");
    }
}
```



③ JVM creates Main thread & load main() in stack.

④ Execute Main.

⑤ Student Obj created

```java
class Student {
    String name;
    int age;
    static String college = "IIT G";

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Instance method (normal method)
    public void markAttendance() {
        System.out.println(name + " is present");
    }

    // Static method
    public static void changeCollege(String newCollege) {
        college = newCollege;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Aditya", 28);

        s1.markAttendance();

        Student.changeCollege("IIT Bombay");
    }
}
```

Types of References

- Strong Reference
- Soft Reference
- Weak Reference.
- Phantom Reference.

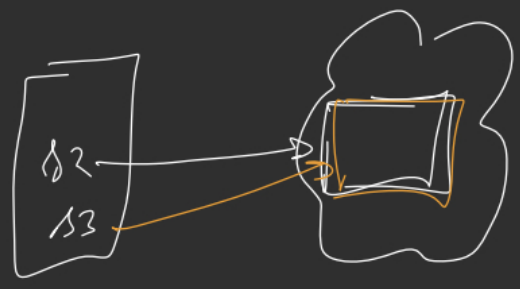
(strong)

(almost gone)

---

(1) Student s1 = new Student();

(2) SoftReference<Student> (s2) = new SoftReference<>();



s2.get()
→ null.

(3) WeakReference<Student> s3 = new WeakReference<>();

↓

WeakHashMap

(4) phantomReference :

(s4).get() == null

Garbage Collection

- Mark & Sweep
- Mark Compact
- Copying.

Mark & Sweep (Major GC)

Heap: ✓ ✗ ✓ ✗ ✓

[A] [B] [C] [D] [G]

→ [A] [ ] [C] [ ] [E]

Mark & Compact (Major GC)

[A] [B] [C] [D] [E]

[A] [C] [E] → No fragmentation

→ Moving obj is costly

Copying [Minor GC]

Eden:

~~[A] [B] [C] [D]~~

So

~~[B] [D]~~

S1

[B] [D]

Garbage Collection

↳ It 'Stop the World' (STW)

System.gc()

GC

Sequential GC

Parallel GC

↳ Multiple threads

Out Of Memory Error

↳ Heap Dump

Visual VM